

Fleet Tracking Platform - Frontend Take Home Assignment

Problem Statement

You are building the web interface for a fleet tracking platform.

Mock API Setup

Important: There is no backend provided. You need to handle data on your own.

Your options:

Option 1: Mock API Tools (Recommended)

Use any tool you're comfortable with:

- json-server - Create `db.json` with your data structure
- MSW (Mock Service Worker) - Mock at network level
- Mirage JS - In-memory API mocking

Option 2: Simple Local Backend

Build a minimal Express/Node server with in-memory data if you prefer.

Option 3: Frontend-Only State

Use React Context, Redux, or Zustand with hardcoded initial data (not recommended, but acceptable).

What we expect:

- You create realistic sample data (5-10 hubs, terminals, drivers, vehicles, orders)
- Your app demonstrates all required functionality
- Data persists during the session (doesn't need to survive page refresh)

We're evaluating your frontend skills, not your ability to build backends. Pick the fastest approach that lets you focus on UI/UX.

Sample Data Structure

To help you get started, here's what your data should look like:

Hubs/Terminals:

```
{
  "id": "hub-1",
  "name": "Downtown Distribution Hub",
  "type": "hub",
  "address": "123 Main St, City",
  "coordinates": { "lat": 40.7128, "lng": -74.0060 },
  "inventory": {
    "diesel": 15000,
    "petrol": 12000
  }
}
```

Drivers:

```
{
  "id": "driver-1",
  "name": "John Smith",
  "license": "DL-123456",
  "phone": "+1-555-0100"
}
```

Vehicles:

```
{
  "id": "vehicle-1",
  "registration": "TRK-101",
  "capacity": 8000,
  "type": "Tanker"
}
```

Orders:

```
{
  "id": "order-1",
  "destinationId": "terminal-5",
```

```
"product": "diesel",  
"quantity": 5000,  
"deliveryDate": "2025-11-24",  
"assignedDriverId": "driver-1",  
"status": "assigned"  
}
```

Create enough sample data to demonstrate all features realistically.

What You're Building

1. Admin Dashboard

Pages Required:

Master Data Management

- Forms to create/edit: Hubs, Terminals, Products, Drivers, Vehicles
- List views with search and filters
- Validation and error handling

Order Management

- Create order form (select destination, product, quantity, date)
- Assign orders to drivers
- View all orders with status filters

Vehicle Allocation

- Interface to allocate vehicles to drivers for specific dates
- Calendar view showing vehicle allocations
- Prevent double-booking (show error if vehicle already allocated)

Live Fleet Map (Core Feature)

- Map showing all active vehicles with real-time locations
- Vehicle markers with driver info tooltip
- Filter by driver, vehicle, or delivery status
- Auto-refresh every 30 seconds (or manual refresh button)

Inventory Dashboard

- Table showing all hubs/terminals with product inventory
- Color coding for low stock alerts
- Search and filter capabilities

2. Driver Interface

Shift View

- Card showing today's shift details (vehicle, orders)
- Button to "Start Shift" (disabled if no allocation)
- List of assigned deliveries with destination and quantities

Live Map

- Show driver's current location (simulated)
- Show destination markers
- Route line (optional)
- Button to "Send GPS Update" (simulates location tracking)

Delivery Management

- For each delivery:
 - Mark as Completed (shows success message, updates inventory)
 - Mark as Failed (shows reason input modal)
- Button to "End Shift"

Shift History

- List of past shifts with deliveries completed

Technical Requirements

Must Have

- React, Vue, or Angular
- TypeScript (preferred) or JavaScript
- Map integration (Leaflet.js or Mapbox)
- Responsive design (mobile and desktop)
- Component-based architecture
- State management (Context API, Redux, Vuex, etc.)

Code Quality

- Clean, reusable components
- Proper prop types / TypeScript interfaces
- Error boundaries
- Loading states
- Form validations
- Consistent styling

Testing

- Unit tests for utility functions
- Component tests for key features
- Integration tests for critical user flows
- Mock API calls in tests

Nice to Have

- Toast notifications for success/error
- Skeleton loaders
- Optimistic UI updates
- Dark mode
- Accessibility (ARIA labels, keyboard navigation)
- Animation/transitions

Mock API Approach

You have two options:

1. Use provided mock responses (we'll give you JSON files)
 - Use json-server or msw (Mock Service Worker)
 - Simulate API delays and errors
2. Build simple backend (if you're comfortable)
 - Node.js + Express with in-memory data
 - Just enough to support frontend features

Deliverables

1. Code Repository

- Clean component structure
- README with setup instructions
- Package.json with all dependencies

2. Documentation

- docs/COMPONENTS.md - Component hierarchy and responsibilities
- docs/STATE_MANAGEMENT.md - How you manage application state
- docs/DECISIONS.md - Technical choices (why this library, pattern, etc.)

3. Demo

- Video recording (2-3 minutes) showing key workflows:
 - Admin creates order and allocates vehicle
 - Driver starts shift and completes delivery
 - Admin views fleet map and inventory updates
- Or deployed version (Vercel, Netlify)

4. Testing

- Test coverage report
- Instructions to run tests

Key User Flows to Implement

1. Admin Flow: Create hub → Create product → Create order → Allocate vehicle → View on map
2. Driver Flow: Start shift → View deliveries → Send GPS updates → Complete delivery → End shift
3. Real-time Updates: Admin sees driver location update on map
4. Error Handling: Try to allocate same vehicle twice (should show error)

What We're Evaluating

- UI/UX design and usability

- Component architecture and reusability
- State management approach
- Code organization and cleanliness
- Handling of async operations and loading states
- Error handling and edge cases
- Responsiveness and performance
- Testing quality

Time Expectation

This assignment should take 2-3 days. Please refer to the email for the exact submission date.

Submission

Share GitHub repo link or grant access to:

- jitendra@fleetpanda.com
- neranjen@fleetpanda.com